TEAM WORLD CHANGER

SMART DISCUSSION FORUM

SOFTWARE DESIGN DOCUMENT

https://github.com/luwi3/SMART-DISCUSSION-FORUM

G-12

SUPERVISOR: MR. LULE EMMANUEL

Table 1 GROUP MEMBERS

NAME	REGNO
OPOYA ZURUBABEL	25/U/04314/PS
KISAKYE EMMANUEL	25/U/04233/PS
NDAGIRE DOROHTY	25/U/04296/PS
NABBUMBA KAREN	25/U/04276/PS
NSUBUGA ELIJAH SOLOMON	25/U/0390

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the high-level architecture and system design for the Educational Discussion Forum and Quiz Platform. The primary purpose of this document is to provide a comprehensive engineering blueprint that translates the 12 core functional and behavioral requirements into technical designs. The intended audience includes the development team for implementation guidance, system administrators for tracking deployment pipelines, and academic stakeholders verifying administration controls.

1.2 Scope

The software product being developed is a hybrid Web and Desktop Educational Forum and Automated Quiz Platform.

Goals and Objectives: The primary objective is to solve engagement, monitoring, and clutter issues faced by academic discussion groups. The system aims to minimize forum clutter, provide robust offline capability, automate student engagement tracking, ensure locked-down academic integrity during assessments, and use artificial intelligence to categorize and recommend content dynamically.

Benefits: Delivers cross-platform chat access with offline capabilities for students, automatic grading and participation tracking for lecturers, and an automated warning/blacklisting pipeline for administrators to curb disruptive behavior automatically.

1.3 Overview

This document is organized into eight distinct logical segments to systematically guide the reader through the system framework:

Section 1 (Introduction): Outlines the structural background, system boundaries, document scope, and operational definitions.

Section 2 (System Overview): Delivers a high-level conceptual summary of the system context, user roles, and main functionalities.

Section 3 (System Architecture): Formally maps out the multi-tier Server Tier design, object-oriented decompositions, database drivers, and specific engineering rationales based on the architectural diagram.

Section 4 (Data Design): Details the transformation of the system's information domain into data structures and provides a complete data dictionary of objects.

Section 5 (Component Design): Takes a closer look at what each class component does by mapping their member functions into Procedural Description Language (PDL) or pseudocode.

Section 6 (Human Interface Design): Explains the system from the user's perspective, mapping out screen workflows, user interactions, and visual layouts.

Section 7 (Requirements Matrix): Formally cross-references and traces the designed software components and data structures directly back to the 12 core requirements.

Section 8 (Appendices): Contains any optional or supplementary reference materials.

1.4 Reference Material

GeeksforGeeks. (n.d.). Unified Modeling Language (UML) introduction. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/

Richards, M. (2015). Software architecture patterns. O'Reilly Media.
Interaction Design Foundation. (n.d.). Interaction design and user experience resources.Interaction Design Foundation

Material Design. (n.d.). Material Design guidelines. Material Design

Figma. (n.d.). Figma design resources and community templates.Figma
For database v. Database system a practical approach to design implementation and Management 4th Edition by Thomas Connelly Carolyn Begg

1.5 Definitions and Acronyms

The following definitions, acronyms, and abbreviations are used throughout this SDD:

Table 1 definitions and acronyms

Term / Acronym	**Definition**
SDD	Software Design Document.
RDBMS	Relational Database Management System; the central database holding structured tables (MySQL/PostgreSQL).
Local Cache	A lightweight, locally hosted database instance (SQLite) inside the desktop application bundle for offline message access.
ML Engine	Machine Learning Engine; the asynchronous service tracking text features to handle automatic topic classification and

	recommendations.
API	Application Programming Interface; standard communication protocols allowing servers, client apps, and social platforms to exchange data.

2. SYSTEM OVERVIEW

The platform functions as a specialized, highly moderated hybrid of an asynchronous messaging board and an interactive academic assessment engine. It addresses 12 critical user experiences and challenges categorized across four major operational vectors:

1. Advanced Moderation & User Management

The system provides administrators with automated oversight tools. To combat message flooding and irrelevant submissions, the application monitors member behavior. Inactive users or individuals flouting rules receive automated warning notifications. If a user fails to comply within a set time, the platform automatically places them on a configured temporary blacklist. Furthermore, an onboard rule-acceptance gateway requires newly joining members to review and accept platform criteria before finalizing their registration.

2. Multi-Channel Chat & Offline Synchronization

Users can interface with the platform through responsive web standard views or via native desktop client applications. To handle intermittent internet connectivity, the desktop platform features an isolated caching mechanism. When a user loses internet access, the desktop application transitions to an offline mode, letting them browse previously downloaded history and write new drafts locally. Once network connectivity returns, a background synchronization engine automatically pushes pending data to the cloud database and fetches new messages.

3. Academic Quizzes & Automated Grading

The application provides lecturers with a dedicated configuration dashboard to post and monitor academic quizzes. Lecturers can define time, date, duration, and target student categories. When a quiz goes live, the client app interfaces switch to a locked view that limits access to outside applications. The platform handles forced auto-

submissions the moment countdown timers expire, ensuring fairness even if a student joins late. Following quiz completion, the platform generates comprehensive performance reports and automatically tallies participation scores based on forum interaction frequencies.

4. Intelligent Curation & Social Integrations

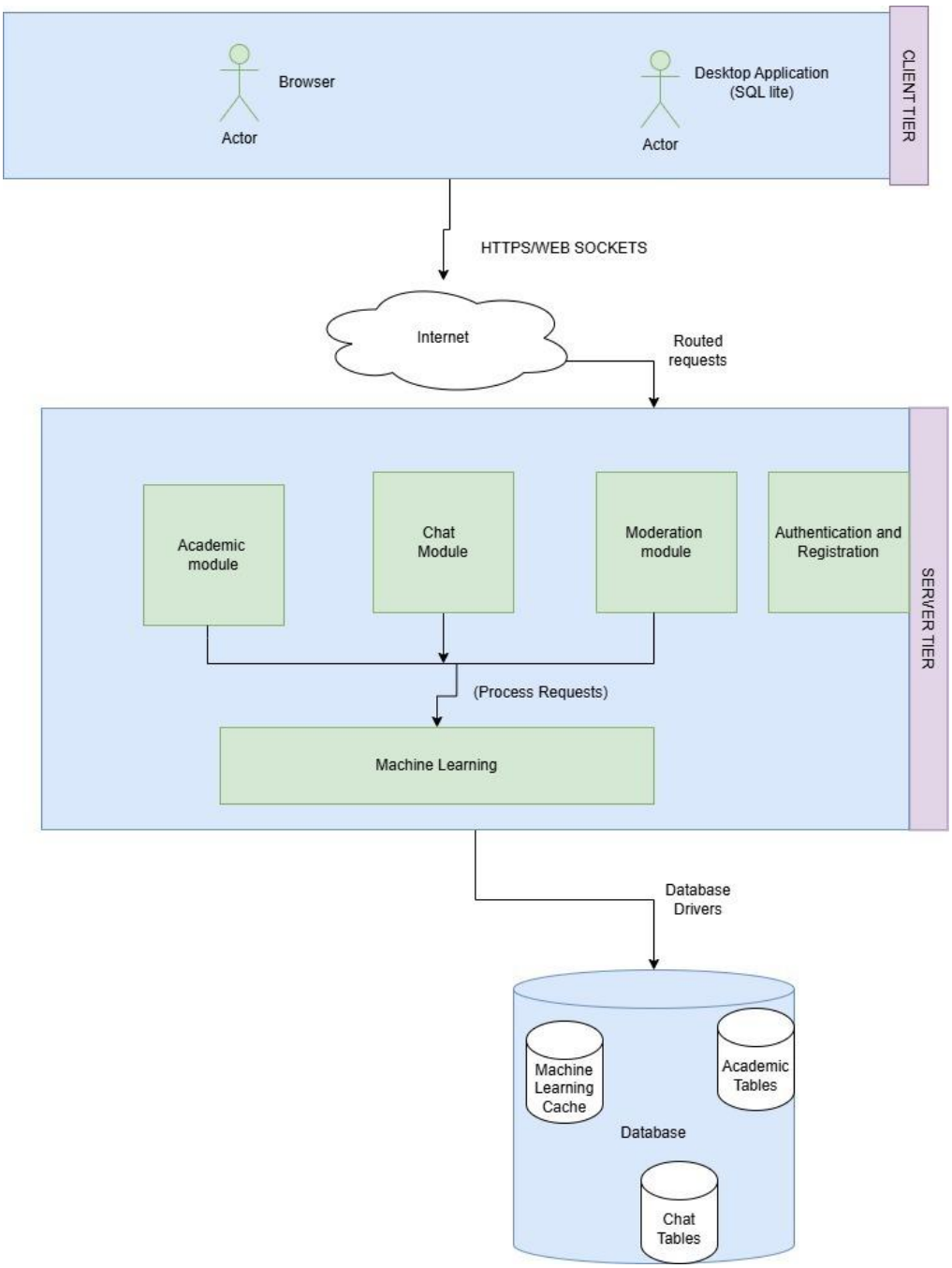
An isolated machine learning engine works continuously in the background to automatically categorize discussion topics and filter threads. By tracking user engagement, the system generates custom recommendation feeds to help users find relevant conversations. Additionally, a social media gateway allows authorized users to forward relevant forum posts directly to external third-party social media channels.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

The system implements a centralized **Server-Tier Architecture** that handles inbound external client requests via the internet, processes them using decoupled functional modules, and relies on a unified data layer for persistence.

This multi-tiered layout separates interface communication, business logic execution, and database driving layers.



th

Figure 1.0 The Architectural diagram

3.1.1 Subsystem and Module Breakdown

The system is decomposed into three distinct layers as illustrated in the architectural diagram:

1. Communication Gateway (Network Layer)

The Internet Engine: Acts as the entry point for all client requests. It captures user interactions from both the web and desktop clients and routes them directly down into the specialized processing modules within the core server boundary.

2. Core Server Tier (Application Logic Layer)

The server tier isolates distinct domain responsibilities into standalone processing modules to maximize cohesion and simplify system maintenance:

Academic Module: Manages educational features, tracking course data, configurations, and transactional components related to quizzes and evaluations.
Chat Module: Governs live and synchronous message passing, topic thread distribution, and real-time interaction pipelines.
Moderation Module: Evaluates platform usage logs, monitors user registration compliance rules, and handles automated penalty actions like blacklisting.
Machine Learning Engine: Placed as a shared processing utility beneath the primary logic handlers. The Academic, Chat, and Moderation modules interact with the Machine Learning subsystem using high-speed **Internal Function Calls** to automatically classify topics and generate user recommendation matrices on demand.

3. Data Tier (Persistence Layer)

Database Drivers: Serves as the connectivity layer that abstracts database communication protocols. It safely pipelines structural queries generated by the Server Tier down into the raw storage blocks.

Unified Relational Database: A central data store that keeps system segments cleanly isolated into dedicated schemas:

Academic Tables: Persists quiz scores, configurations, and performance report structures.

Chat Tables: Stores historical forum messages, user mapping indexes, and topic metadata.

Machine Learning Cache: Holds processed engagement logs, user interaction statistics, and pre-computed recommendation tables to accelerate content loading.

3.1.2 Component Relationship and Control Flow

Inbound Ingestion: User actions pass over the **Internet** directly to target destinations within the **Server Tier** (e.g., a student sending a forum message hits the *Chat Module*).

Internal Intelligent Augmentation: When processing complex content, the primary modules invoke the **Machine Learning** block via synchronized **Internal Function Calls** to parse text patterns or access user engagement profiles.

Persistent Commitment: Once calculations conclude, data is systematically mapped through native **Database Drivers** to be written cleanly into either the *Academic Tables*, *Chat Tables*, or the *Machine Learning Cache* for long-term storage and quick future retrieval.

3.2 DECOMPOSITION DESCRIPTION

3.2.1 Subsystem Architecture and Interface Dependency Model

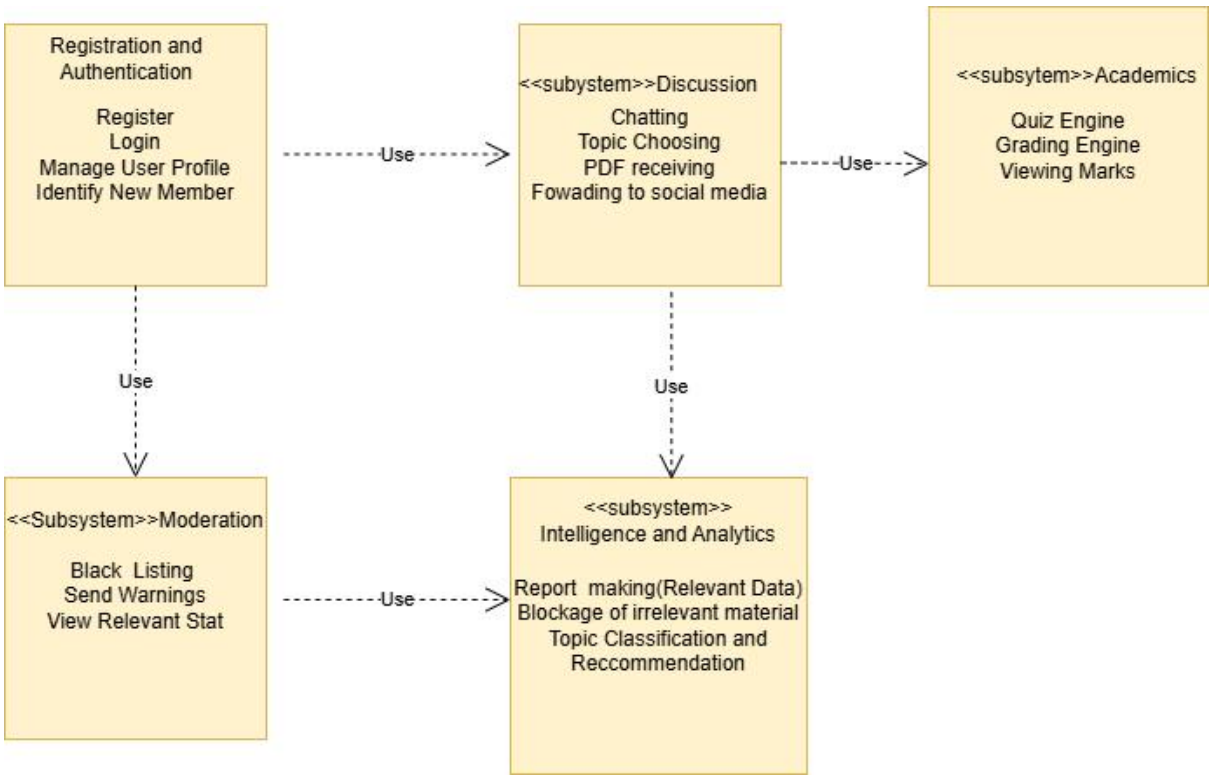


Figure 2 The description of subsystem architecture

3.2.2 Architectural Overview

This structural diagram illustrates the modular architecture of the proposed Learning Management & Collaboration System. The design is partitioned into five core functional subsystems mapped from the operational requirements : Registration and Authentication, Discussion, Academics, Moderation, and Intelligence and Analytics.

3.2.3 Functional Breakdown of Subsystem Components

Subsystem 1: Registration and Authentication

This subsystem serves as the security and identity foundation for the platform. It exposes critical user-lifecycle services:

- Register / Login: Implements the onboarding logic (registerUser()) and handles secure token-based session provision (loginUser()).
- Manage User Profile / Identify New Member: Facilitates the mutation of user profile metadata and systemic permissions (manageUserProfile()).
- Outgoing Dependencies: This subsystem points down to Moderation to feed identity contexts for compliance, and points right to Discussion to ensure all active communication threads contain authenticated session payloads.

Subsystem 2: Discussion

This component serves as the application's real-time communication tier. It contains the operational logic for user interaction:

- Chatting / Topic Choosing: Dispatches live chat message streams (sendMessage()) and applies contextual filters based on selected topic scopes (chooseTopic()).
- PDF receiving / Forwarding to social media: Orchestrates content handling and handles data export pipelines to external APIs (forwardToSocial()).
- Outgoing Dependencies: It points right to Academics (linking chat contexts to learning spaces) and points down to Intelligence and Analytics (streaming raw text data for machine learning evaluation).

Subsystem 3: Academics

This module encapsulates the core educational evaluation engine and grade tracking infrastructure.

- Quiz Engine: Controls the configuration (setQuiz()) and extraction (requestQuizData()) of evaluation structures.
- Grading Engine: Handles runtime response submission processing (submitAnswers()), drives automated scoring algorithms (calculateScore()), and persists records to grade storage (saveResult()).
- Viewing Marks: Exposes secure retrieval interfaces for student grade profiles (viewMarks()).

Subsystem 4: Moderation

This boundary enforces compliance and administrative authority across the platform environment.

- View Relevant Stat: Pipes behavior alerts into administrative interfaces to monitor system health (receiveRelevantStat()).
- Send Warnings / Black Listing: Executes formal policy enforcement actions and revokes infrastructure privileges for target accounts (sendWarning(), blacklistUser()).
- Outgoing Dependencies: It points right to Intelligence and Analytics, indicating that the moderation workflows rely directly on data-driven telemetry to trigger admin alerts.

Subsystem 5: Intelligence and Analytics

This subsystem functions as the background analytical engine, utilizing baseline machine learning models to optimize and secure the platform environment.

- Report making (Relevant Data): Compiles trend evaluations to generate system statistics (provideRelevantStat()).
- Blockage of irrelevant material: Runs automated keyword filtering over text streams to intercept policy violations (blockIrrelevantMat()).

- Topic Classification and Recommendation: Categorizes content streams semantically and generates user feed targeting arrays (classifyTopics(), recommendTopics()).

5. System Interface Specifications

This component of the documentation defines the exact function signatures, input parameters, and expected return types across the five core operational modules. It serves as a contract between subsystems, ensuring that data is passed predictably and securely throughout the application lifecycle.

System Interface Specifications

Table 2 This document shows the interfaces and functions for the system

Interface Name (Module)	Implementers (Functions)	Core Responsibility
IAuthenticationOnboarding	detectNewUsersOnboarding() displayRules() verifyAgreementInput() assignUserRole()	Handles new users when they join. It shows the rules, checks if the user agrees to them, and sets their role as a student or lecturer.
IServerChatModule	isolateTopicThreads() applyExclusionFilter() forwardToSocialMedia() recordPostTimestamp() cleanTextTokens() (AI) classifyAcademicTopic() (AI) filterIrrelevantSpamBlockages() (AI)	Manages the chat rooms and separates different topics. It saves the time of each post and uses AI to clean up text, find the topic, and block spam.
IServerAcademicModule	configureQuizSchedule() verifyExamEligibility() enforceDesktopScreenLock() checkQuizCountdownTimeout() evaluateAutoSubmission() awardPerformanceMarks()	Handles tests and quizzes. It checks who is allowed to take a test, locks the screen to prevent cheating, runs the timer, submits answers automatically when time is up, and gives scores.
IServerModerationModule	detectNewUsersOnboarding() auditLastActivityTimestamp()	Monitors what users are doing. It

	issueInactivityWarnings() executeAccountBlacklisting() verifyInternetConnectionStatus() reconcileDatabaseSync()	checks how long users are active, sends warnings if they are away too long, blocks bad accounts, checks internet connection, and updates data.
IDatabaseStorageSync	secureDatabaseDrivers() writeToChatTables() writeToAcademicTables() updateMachineLearningCache()	Saves all information safely. It writes chat messages and test scores into the database tables and updates the AI memory.

3.2.4 OBJECT DIAGRAMS

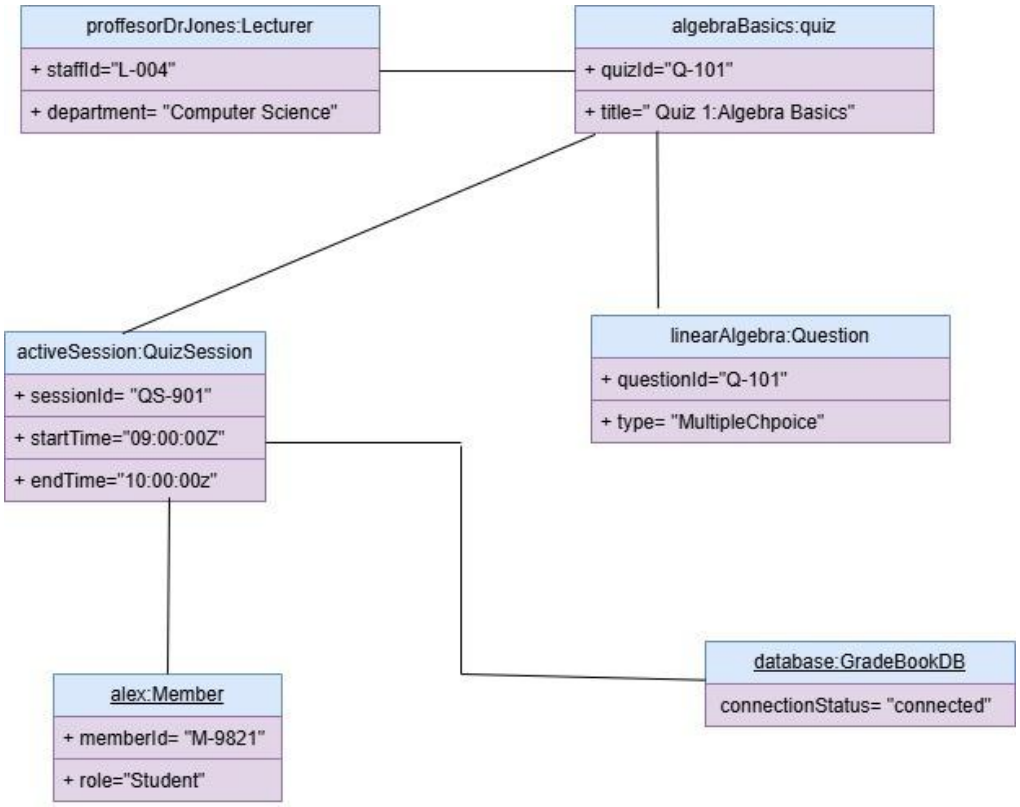


Figure 3 Snap of a chatting instance

Description of the chatting instance

The above object diagram shows how different objects interact in order to fulfill the chatting with in the Alex is a member who makes a post but every post belongs to a particular topic . then the active chat session first saves the messages before handing them to the machine learning instance

The Quiz Instance

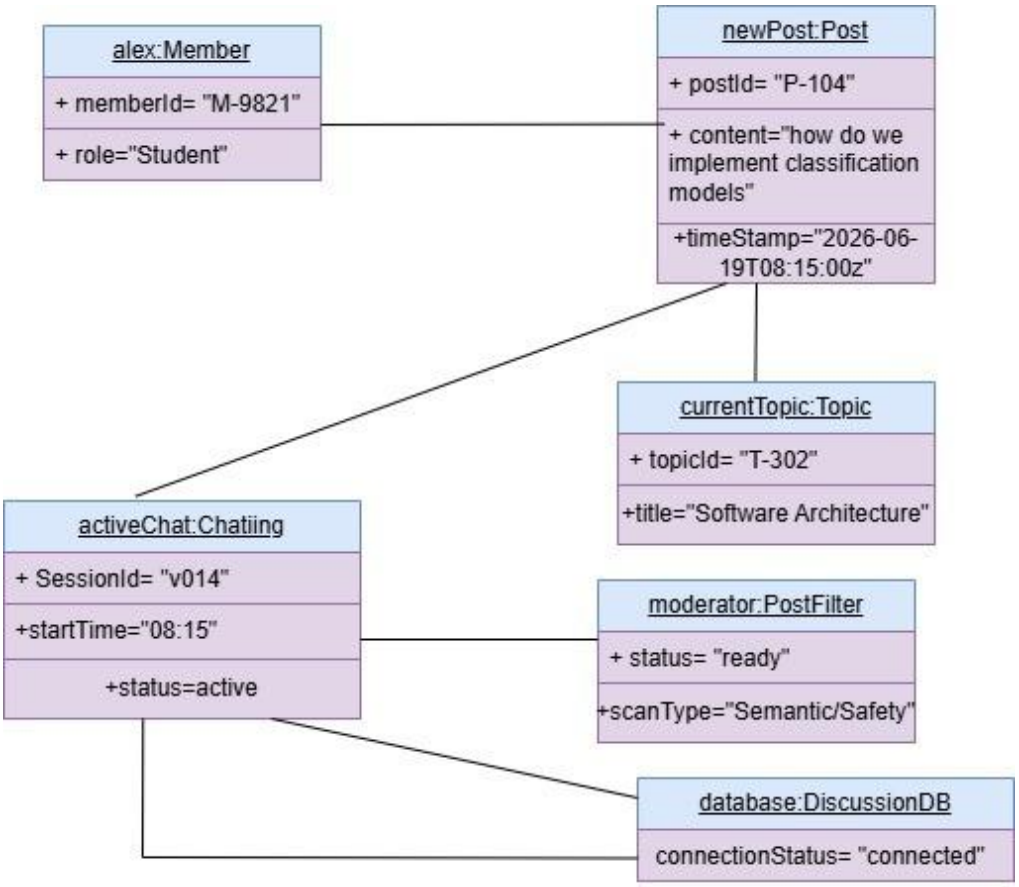


Figure 4 A snap of the quiz session

Description of the quiz session

The diagram shows how the object instances interact to make the quiz session a success. The lecturer makes the quiz but the quiz has questions as entities , the active quiz session either makes the quiz autmatically pop or the student goes and gets it but all that data is being stored in the database

3.2.5 Sequence Model

1. Architectural Overview

This sequence diagram illustrates the runtime interaction and behavioral flow for the quiz evaluation lifecycle within the Academics subsystem. The model follows the Model-View-Controller (MVC) or Robustness analysis pattern, mapping communication between an actor, boundary elements, control components, and data repositories over a chronological timeline.

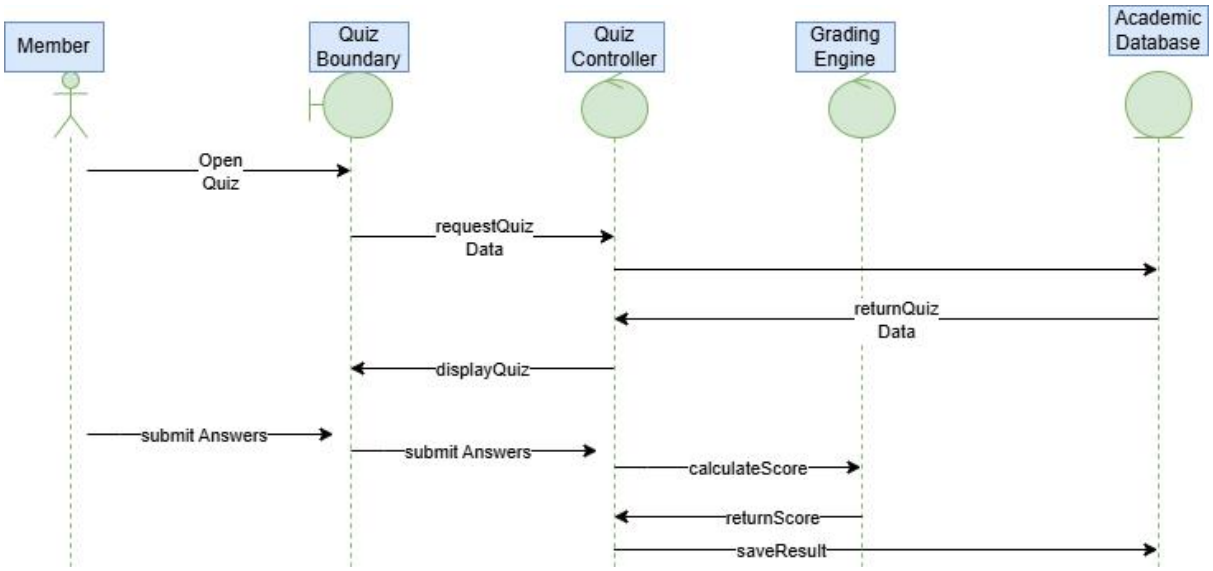


Figure 5 Snapshot of the quiz session

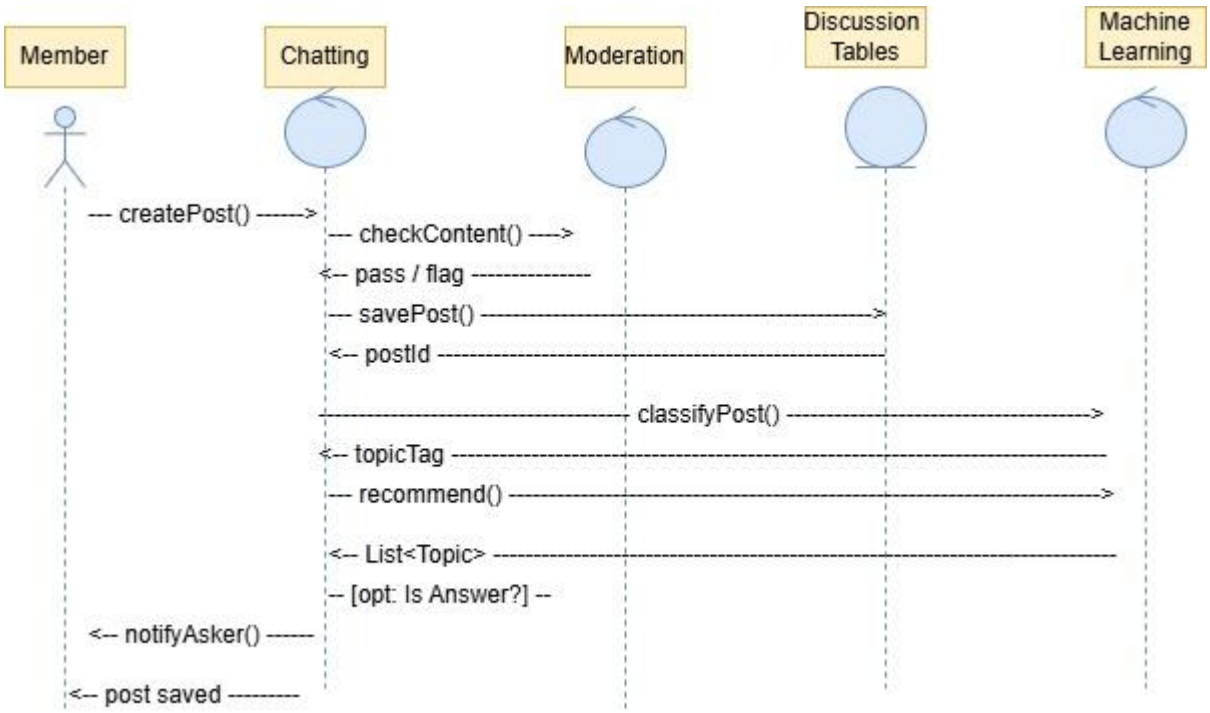


Figure 6 Chatting Module Sequence

Diagram Description

The Chatting Module Sequence Diagram illustrates the runtime interactions and message flows between components during the creation, validation, storage, and intelligent enrichment of a post within the Discussion Subsystem.

The workflow follows a standard Controller-Repository pattern, supplemented by external automated moderation and intelligence services to decouple core business logic from data analysis tasks.

3.3 Design Rationale

3.3.1 Architectural Selection and Requirement Mapping

Hybrid Web/Desktop Distribution Architecture (Requirements 8 & 10): A centralized system framework is paired with a local engine block (SQLite + Caching layer). This guarantees users continuous desktop read access to locally stored history when disconnected from network resources. When online, WebSockets allow real-time notifications to block the client application window instantly upon a quiz launch event.

Asynchronous Isolated Machine Learning Workers (Requirement 11): Heavy calculations—such as running natural language classification and rendering user engagement-driven thread recommendations—are isolated onto an independent machine learning script worker framework. This keeps processing loads away from the main message processing stream to maintain fluid user chat responses.

Abstract Gateway Adapter Layer (Requirement 12): Due to fluid changes in external third-party authorization schemas and endpoints, social platform sharing uses an **Adapter Interface Design Pattern**. All custom interactions map down into generic payloads, letting developers append new sharing endpoints later without changing backend structures.

3.3.2 Critical Design Bottlenecks and Trade-offs

Strict Central Server Control vs. Offline Independence: To avoid unauthorized state drift, user authentication tracking (Requirement 4) and grading matrices evaluation (Requirements 9 & 10) are handled strictly server-side. Desktop local autonomy is intentionally restricted to read-only browsing and local drafting pipelines.

Consistency Constraints on Auto-Submissions: To eliminate client side latency manipulation or malicious code changes during assessment windows, late user attempts are limited using an authoritative system timestamp counter check. When a quiz time parameter expires, database operations lock down the user's specific quiz profile instantly, preferring strict server-side state security over client processing independence.

Alternative Paradigm Consideration: A distributed Peer-to-Peer messaging matrix framework was evaluated to minimize server operating overhead. However, this structure was rejected. Central structural authorization anchors are necessary to process platform activity logging, compute individual engagement scores, manage

administrative moderation pipelines, and guarantee isolated security context frameworks during quiz examinations.

4 DATABASE DESIGN

4.1 Data Description

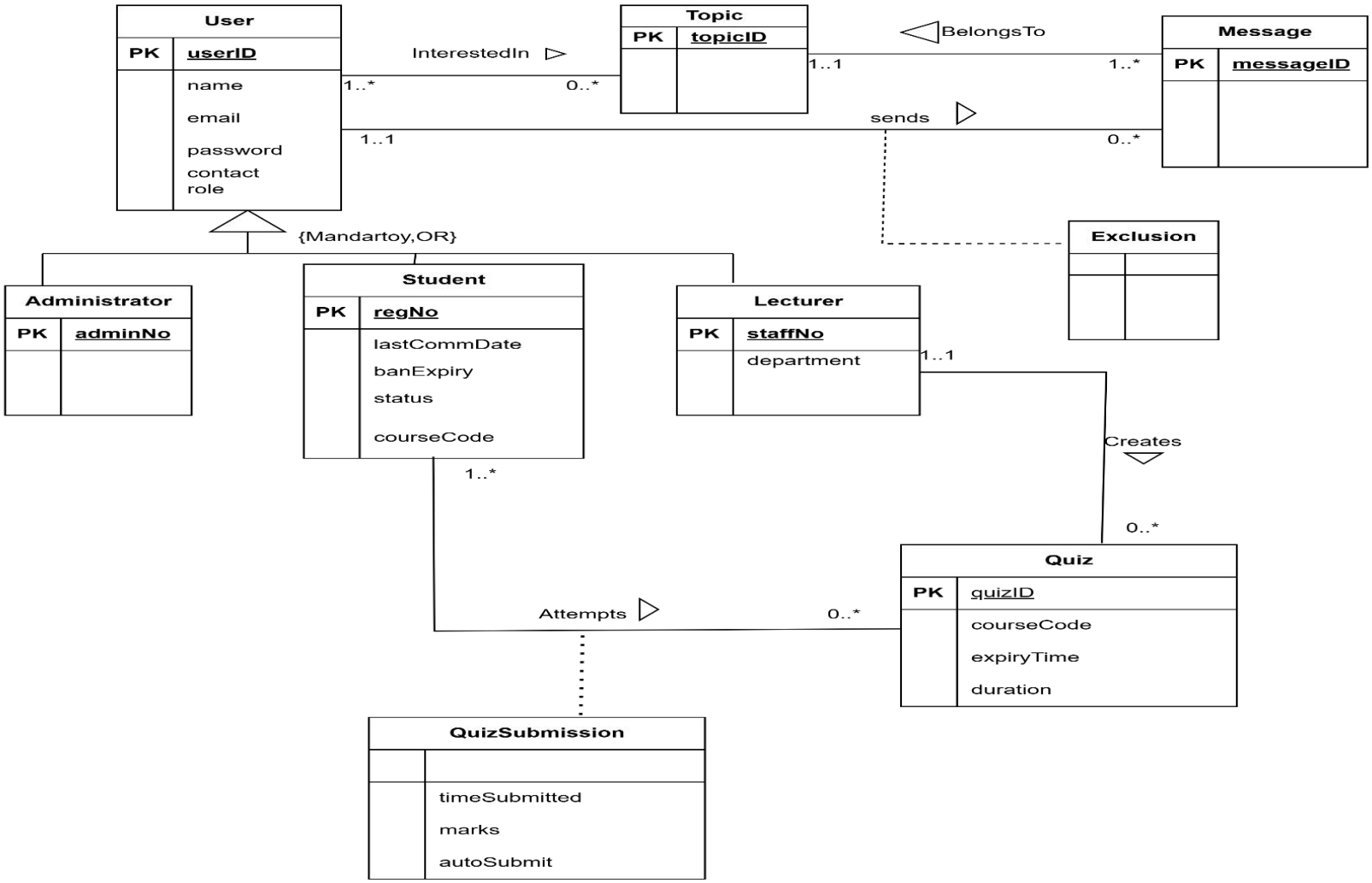
The Smart Discussion Forum system will use a relational database to store and manage information related to users, discussions, quizzes, participation records, notifications, and administrative activities. The database is designed to support role-based access for Administrators, Lecturers, and Students while maintaining data integrity and consistency.

The database design is based on the Enhanced Entity Relationship Diagram (EERD). The User entity acts as a supertype from which the Administrator, Lecturer, and Student entities are derived. This approach reduces redundancy while supporting role-based access control.

Discussion activities are managed through Topic and Message entities. Users can participate in topics, send messages, and indicate interests in particular discussion topics.

The quiz subsystem is implemented using the Quiz and QuizSubmission entities. Lecturers create quizzes while students attempt them. Quiz submissions store marks, submission time, and auto-submission status.

The Exclusion entity supports selective communication by allowing users to exclude specific members from particular communications. Student participation is monitored using attributes such as last communication date, status, and ban expiry date to facilitate warning and blacklisting mechanisms


User
PK
userID
name
email
password
contact
role
InterestedIn
1..*
0..*
Topic
PK
topicID
BelongsTo
1..1
1..*
Message
PK
messageID
sends
1..1
0..*
{Mandartoy,OR}
Exclusion
Administrator
PK
adminNo
Student
PK
regNo
lastCommDate
banExpiry
status
courseCode
Lecturer
PK
staffNo
department
1..1
Creates
0..*
1..*
Quiz
PK
quizID
courseCode
expiryTime
duration
Attempts
0..*
QuizSubmission
timeSubmitted
marks
autoSubmit

4.2 Data Dictionary

Table 3**Administrator**

Field Name	Data Type	Constraint	Description
adminNo	VARCHAR(15)	Primary Key	Unique identifier for an administrator
admin_userID	VARCHAR(15)	Foreign Key	References the associated user account

Table 4**Lecturer**

Field Name	Data Type	Constraint	Description
staffNo	VARCHAR(15)	Primary Key	Unique lecturer staff number
department	VARCHAR(255)	NOT NULL	Lecturer's department
userID	VARCHAR(15)	Foreign Key	References the associated user account

Table 5 Student

Field Name	Data Type	Constraint	Description
regNo	VARCHAR(15)	Primary Key	Unique student registration number
courseCode	VARCHAR(15)	NOT NULL	Programme or course code
lastCommDate	DATE	NOT NULL	Date of the student's last communication
status	VARCHAR(25)	NOT NULL	Indicates whether the student is active or blacklisted or on warning
banExpiry	DATE	NULL	Date when the blacklist period expires
userID	VARCHAR(15)	Foreign Key	References the associated user account

Table 6 **User**

Field Name	Data Type	Constraint	Description
userID	VARCHAR(15)	Primary Key	Unique identifier for a user
name	VARCHAR(255)	NOT NULL	Full name of the user
email	VARCHAR(255)	NOT NULL	User email address
password	VARCHAR(25)	NOT NULL	User login password
role	VARCHAR(255)	NOT NULL	User role (Administrator, Lecturer, or Student)
contact	VARCHAR(20)	NOT NULL	User contact number

Table 7**Topic**

Field Name	Data Type	Constraint	Description

topicID	VARCHAR(15)	Primary Key	Unique identifier for a discussion topic

Table 8 **Message**

Field Name	Data Type	Constraint	Description
messageID	VARCHAR(15)	Primary Key	Unique identifier for a message
topicID	VARCHAR(15)	Foreign Key	References the topic to which the message belongs
sender_userID	VARCHAR(15)	Foreign Key	References the user who sent the message

Table 9 **UserInterest**

Field Name	Data Type	Constraint	Description
topicID	VARCHAR(15)	Foreign Key	References the topic of interest
userID	VARCHAR(15)	Foreign Key	References the interested user

Table 10 **Exclusion**

Field Name	Data Type	Constraint	Description
excluded_userID	VARCHAR(15)	Foreign Key	User excluded from a communication
messageID	VARCHAR(15)	Foreign Key	Associated message
sender_userID	VARCHAR(15)	Foreign Key	User initiating the exclusion

Table 11 **Quiz**

Field Name	Data Type	Constraint	Description
quizID	VARCHAR(25)	Primary Key	Unique identifier for a quiz
courseCode	VARCHAR(15)	NOT NULL	Course targeted by the quiz
expiryTime	TIME	NOT NULL	Time when the quiz expires
duration	TIME	NOT NULL	Time allocated for completing the quiz
lecturer_userID	VARCHAR(15)	Foreign Key	References the lecturer who created the quiz
staffNo	VARCHAR(15)	Foreign Key	Lecturer staff number
student_userID	VARCHAR(15)	Foreign Key	References a student user
regNo	VARCHAR(15)	Foreign Key	Student registration number

Table 12 **QuizSubmission**

Field Name	Data Type	Constraint	Description

Field Name	Data Type	Constraint	Description
timeSubmitted	TIME	NULL	Time when the quiz was submitted
marks	INT	NULL	Marks obtained by the student
isAutoSubmitted	BOOLEAN	NOT NULL	Indicates whether the quiz was automatically submitted
regNo	VARCHAR(15)	Foreign Key	References the student who attempted the quiz
student_userID	VARCHAR(15)	Foreign Key	References the student user account
quizID	VARCHAR(25)	Foreign Key	References the quiz attempted

5. COMPONENT DESIGN

5.1 DISCUSSION MODULE

```
CLASS Chat
   // Attributes
   VARIABLE sessionID : String
   VARIABLE startTime : DateTime
   VARIABLE status : String

   // Orchestrates the creation and validation process of a post
   METHOD handleSubmittedPost(memberId: String, text: String, topicId:
String)
      // 1. Intercept content and run through safety filter
      VARIABLE isSafe : Boolean = moderator.validateContent(text)

      IF isSafe == FALSE THEN
         RETURN "Post rejected due to policy violation"
      ENDIF

      // 2. Instantiate a new temporary Post entity object
      VARIABLE tempPost : Post = NEW Post(text)

      // 3. Persist the post to the database tier
      VARIABLE generatedId : String = database.savePostRecord(tempPost,
topicId, memberId)

      // 4. Update object with live database ID
      tempPost.setPostId(generatedId)
```

```
        RETURN "Post successfully processed"
    ENDMETHOD
ENDCLASS
CLASS PostFilter
    VARIABLE status : String

    // Evaluates raw text for compliance violations
    METHOD validateContent(rawText: String) : Boolean
        VARIABLE cleanStatus : Boolean = TRUE

        // Local algorithm scans string for flagged keywords
        IF rawText CONTAINS harmful_content THEN
            cleanStatus = FALSE
        ENDIF

        RETURN cleanStatus
    ENDMETHOD
ENDCLASS
 CLASS DiscussionDB
    VARIABLE connectionStatus : String

    // Commits the verified transaction to persistent tables
    METHOD savePostRecord(postObj: Post, topicId: String, memberId:
String) : String
        // Open database transaction context
        EXECUTE SQL "INSERT INTO PostTable (content, topic_id, author_id)
VALUES (...)"

        // Retrieve the auto-incremented primary key
        VARIABLE newId : String = GET_LAST_INSERT_ID()
```

```
        RETURN newId
    ENDMETHOD
ENDCLASS
 CLASS Post
    VARIABLE postId : String
    VARIABLE content : String
    VARIABLE timestamp : DateTime

    METHOD setPostId(id : String)
        THIS.postId = id
    ENDMETHOD
ENDCLASS
 CLASS Member
    VARIABLE memberId : String
    VARIABLE role : String

    // Standard accessor to retrieve the member's unique system identification
    METHOD getMemberId() : String
        RETURN THIS.memberId
    ENDMETHOD
ENDCLASS
CLASS Topic
    VARIABLE topicId : String
    VARIABLE title : String

    // Standard accessor to identify the forum thread category context
    METHOD getTopicId() : String
        RETURN THIS.topicId
    ENDMETHOD
ENDCLASS
```

5.2 QUIZ MODULE

```
CLASS QuizSession
   // Attributes tracking the live state
   VARIABLE sessionId : String
   VARIABLE startTime : DateTime
   VARIABLE endTime : DateTime
   VARIABLE currentMarks : Integer

   // Initializes a new live testing attempt
   METHOD startSession(quizTemplate : Quiz)
      THIS.startTime = GET_CURRENT_TIME()
      // Calculate deadline based on template rules
      THIS.endTime = THIS.startTime + quizTemplate.getDuration()
      THIS.currentMarks = 0
   ENDMETHOD

   // Evaluates a student's answer submission dynamically
   METHOD submitAnswer(questionId : String, selectedOption : String)
      // Local validation logic
      VARIABLE isCorrect : Boolean = mathQuiz.checkAnswer(questionId,
selectedOption)

      IF isCorrect == TRUE THEN
         // Increment the student's running score
         THIS.currentMarks = THIS.currentMarks +
mathQuiz.getQuestionValue(questionId)
      ENDIF
   ENDMETHOD
ENDCLASS
```

```
CLASS Quiz
    // Attributes defining the quiz rules
    VARIABLE quizId : String
    VARIABLE title : String
    VARIABLE deadline : DateTime
    VARIABLE passMark : Integer
    VARIABLE duration : Duration
    VARIABLE questionBank : Map // Maps QuestionID to CorrectAnswer
and Points

    // Returns the total allowed time for the quiz
    METHOD getDuration() : Duration
        RETURN THIS.duration
    ENDMETHOD

    // Verifies if the student's selected option matches the answer key
    METHOD checkAnswer(questionId : String, selectedOption : String) :
Boolean
        VARIABLE correctAnswer : String =
THIS.questionBank.get(questionId).answer
        RETURN (selectedOption == correctAnswer)
    ENDMETHOD

    // Retrieves the point value for a specific question
    METHOD getQuestionValue(questionId : String) : Integer
        RETURN THIS.questionBank.get(questionId).points
    ENDMETHOD
ENDCLASS
CLASS Lecturer
    VARIABLE staffId : String
```

```
    VARIABLE department : String

    // Allows a professor to initialize a new quiz template configuration
    METHOD createQuiz(id : String, title : String, passMark : Integer) : Quiz
        VARIABLE newQuiz : Quiz = NEW Quiz()
        newQuiz.quizId = id
        newQuiz.title = title
        newQuiz.passMark = passMark
        RETURN newQuiz
    ENDMETHOD
CLASS Question
    // Local data representing an individual question's properties
    VARIABLE questionId : String
    VARIABLE prompt : String          // The actual question text (e.g., "What is
2+2?")
    VARIABLE options : ListOfStrings   // Multiple choice options (A, B, C, D)
    VARIABLE correctAnswer : String    // The key to verify against
    VARIABLE points : Integer         // How much this specific question is
worth

    // Verifies if the student's chosen option is the correct one
    METHOD isAnswerCorrect(submittedOption : String) : Boolean
        RETURN (submittedOption == THIS.correctAnswer)
    ENDMETHOD

    // Getter to retrieve the point value for the quiz score calculation
    METHOD getPoints() : Integer
        RETURN THIS.points
    ENDMETHOD
ENDCLASS
```

6. HUMAN INTERFACE DESIGN

6.1 OVERVIEW OF THE SYSTEM

The Smart Discussion Forum system provides an interface based on the different roles that are going to use the system that is; Administrators, Lecturers and Students who are generally categorized as Users.

There are mainly two interfaces through which users interact with the system;

Web interface implemented using Laravel and for real-time access.

Desktop interface implemented using Java GUI and for offline access where messages are automatically synchronized when internet connectivity is restored.

The following functionality is offered by the system basing on user's perspective;

Login interface

This is used by already registered users to access the system by filling in their email or username and password whenever they return to the system for use.

First time Registration interface

This is used by new users to access the system by filling in their personal details like email, name, contact, password and selecting their role from a dropdown list that is Admin, Student, Lecturer. Besides that, the new user is prompted to read through the following forum rules after filling in their details. The user ticks the attached check box to confirm that they have read and understood the rules. Lastly, if the user clicks on the agree and continue button, they are registered fully and a confirmation message pops up on their screen and if the user clicks on the decline button, their registration is cancelled.

Dashboard

Each specific user has a different dashboard depending on their role;

Administrator dashboard

shows statistics of users like students who are active, those on warning and those blacklisted. It shows them visually in form of charts or graphs.

Student dashboard

Enables a student access their marks, participation records, recommended topics, announcements, notifications, their profile and communicate via chatting.

Lecturer dashboard

Enables a lecturer create and configure quizzes, view student details, communicate via chat, and monitor student participation.

Quiz Interface

This enables lecturer set and configure quizzes having title, quiz ID, course code, lecturer's name, start and expiry time, duration and date of doing the quiz.

6.2 Screen Image

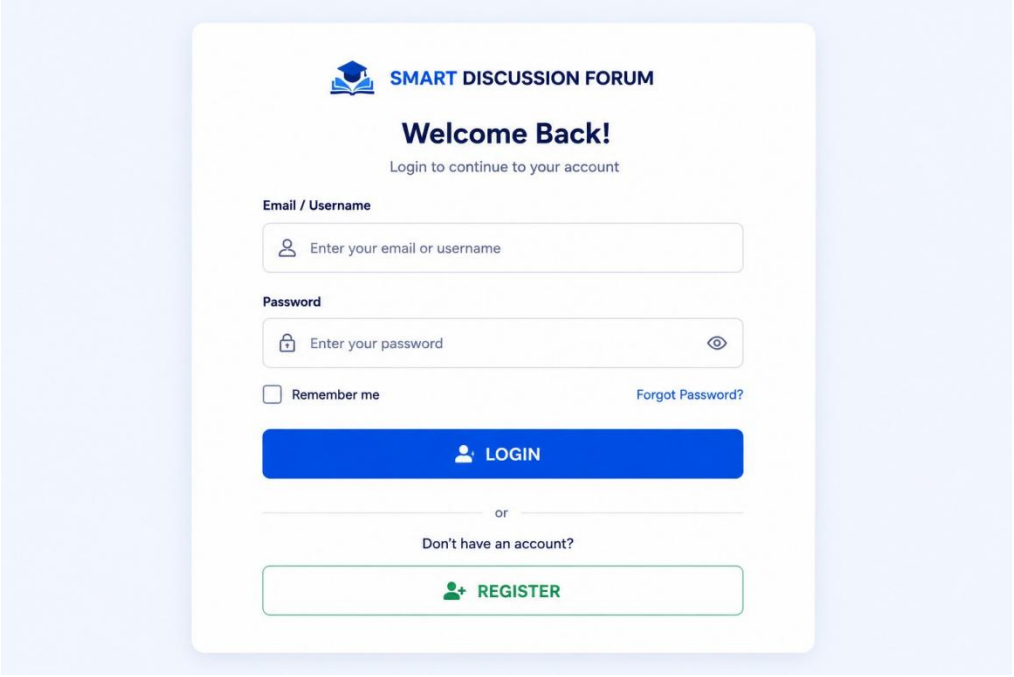
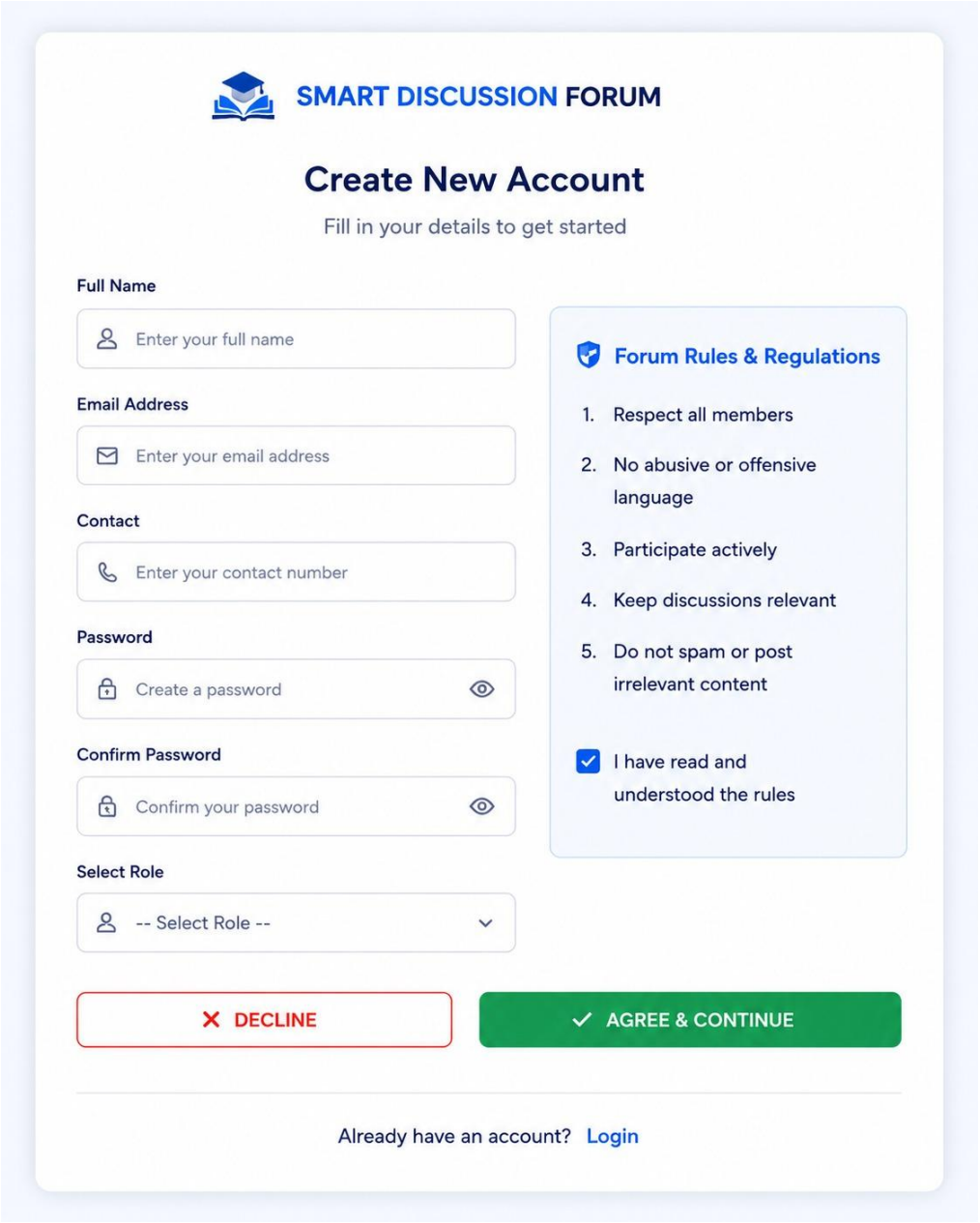


Figure 7 The login screen



b

Figure 8 The first time registration

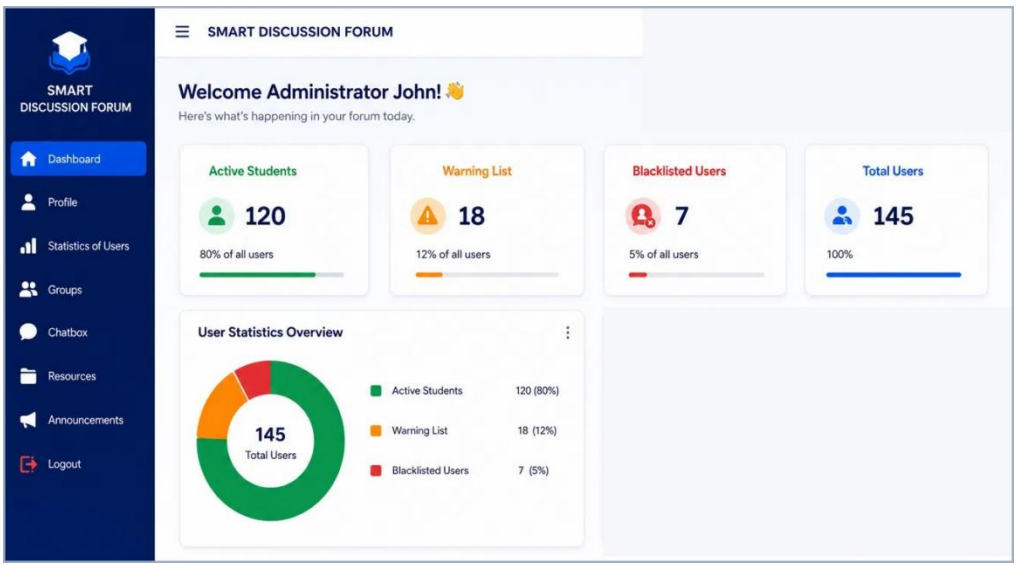


Figure 9 The administration dashboard

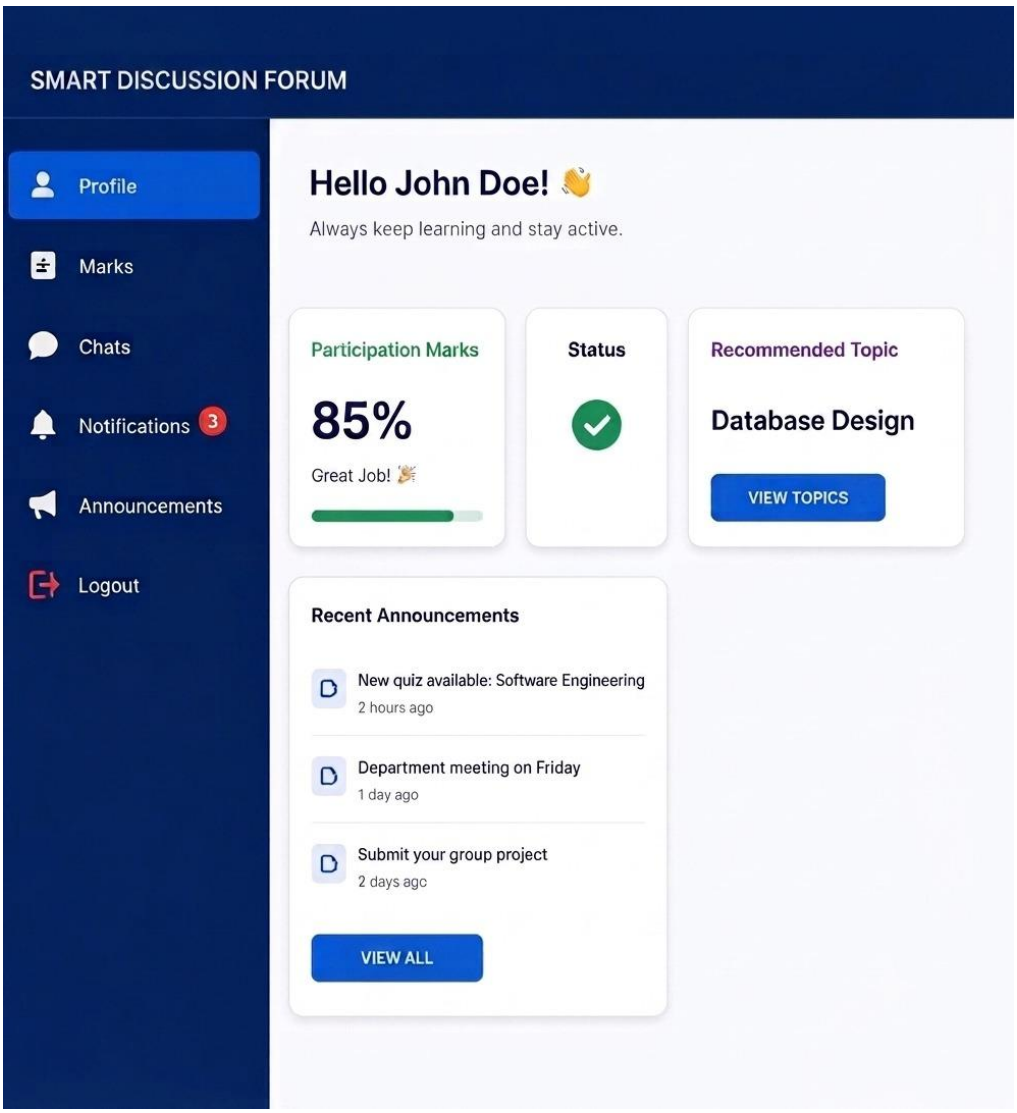


Figure 12 Student's screen

Figure 13 The lecturer dashboard

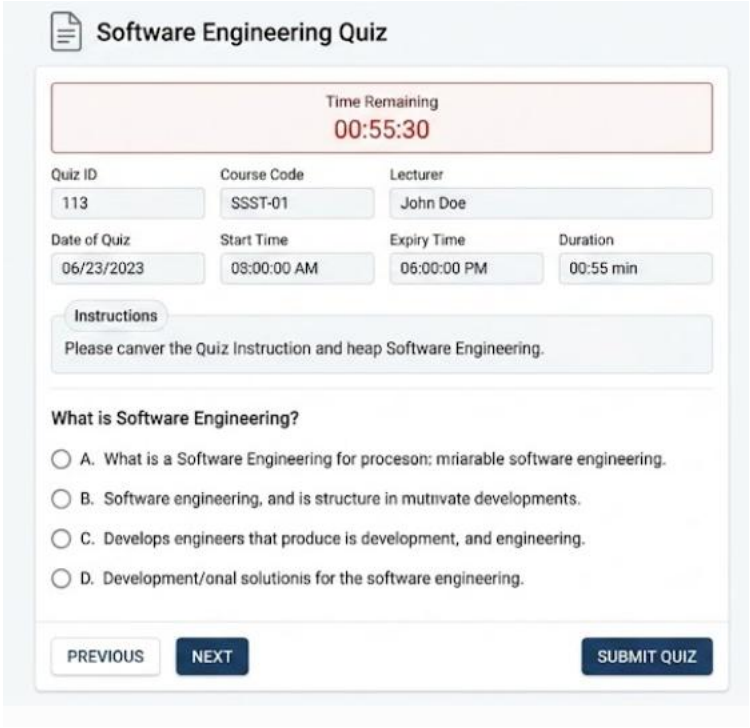


Figure 14 The Quiz interface

6.3 Screen Objects and Action

Table 13 Log in interface

Object	Action
Text fields: Email/Username, Password	User enters credentials → system validates
Buttons: Login, Register, Forgot Password	Forgot Password triggers recovery workflow; "Register" navigates to registration
Label: "Welcome to Smart Discussion Forum	Provides system greeting

Table 14 First- time Registration Screen

Objects	Actions
Text fields: Name, Email, Password, Confirm Password	User fills details
Role selection: Supervisor, Student, Lecturer It is a drop down	User chooses role and it helps them to get the right dashboard
Rules & Regulations box + checkbox "I agree"	Must check "I agree" before continuing
Buttons: Decline, Agree and Continue	"Decline" cancels registration; "Agree and Continue" completes registration

Table 15 Quiz Interface

objects	Action
Labels: Quiz Title, Quiz ID, Lecturer's Name, Course Code, Date of Quiz	Lecturer configures quiz details and are viewed by the students n announcement
Start Time, Expiry Time, Duration	Quiz timing is set and enforcedFields:
Text area: Quiz Instructions	Students view instructions before attempting
Section: Quiz Questions	Students attempt questions; timer runs; auto- submit on expiry

Table 16 Lecturer's dashboard

Sidebar buttons: Profile, Chat, Student Details, Chats, Quiz Setting, Logout	Lecturer navigates via sidebar The profile show the details of lecturer The students details help them see students

	marks in quizzes and also marks in awarded to the due participate
Greeting label: “Hello Lecturer Name”	Displays personalized greeting
Chat	Enables communication with members
Logout	Ends session
Quiz Setting	Opens quiz configuration

Table 17 Student Dashboard

Object	Actions
Sidebar buttons: Profile, Marks, Chats, Notifications, Announcements, Logout	Student navigates via sidebar
Labels: “Hello Student Name,” Status, Recommended Topic, Participation Marks	Displays student info and engagement
Displays student info and engagement	Displays student info and engagement
Chats	Opens discussion forum
Notifications & Announcements	Show updates
Logout	Ends session

Table 18 Administrator Dashboard

Object	Action
Sidebar buttons: Profile, Statistics of Users (Admin, Students, Lecturers), Groups, Chatbox, Resources, Logout	Admin navigates via sidebar
Greeting label: “Welcome Administrator Name”	Displays personalized greeting
Statistics of Users	Admin views statistics per role
Groups	Manages discussion groups
Resources	Provides shared materials
Resources	Provides shared materials
Logout	Ends session